Kasa v2: multi-agent invoicing on RGB

Oct 8, 2026 · @messi

Pitch

Kasa is a self-custodial invoicing app for Italian freelancers and small agencies paid in USDT over RGB, now run by three local agents that each own one job: catching fraud, working out the tax, and one wildcard slot we fill before the event.

- **Fraud Controller** catches invoice and payment discrepancies: wrong totals, duplicates, edited PDFs, under- and overpayments.
- **Commercialista** tracks the Italian tax on every crypto receipt and on every later conversion of USDT into EUR, and tells the merchant how much to set aside.
- **Wildcard** is a third agent slot with a fixed interface; its job is decided by the pre-event deadline (see Agents).

The design rule from v1 still holds and now applies to all three: **agents propose, the merchant approves, code executes.** No agent computes a final amount, changes an invoice state or moves funds on its own. Everything runs on the merchant's laptop; only settlement touches the RGB network.

What changed from v1

The payment flow is the same; the AI layer is split into named agents that plug into it at fixed points, and each teammate now owns one agent end to end.

Area	v1 (Oct 2 proposal)	v2
AI	One generic QVAC assistant	Three named agents on one shared QVAC runtime
Fraud	A risk-flag step after payment	Fraud Controller checks both the invoice draft and the payment
Accounting	Ledger row with EUR value at receipt	Commercialista: income at receipt, gain or loss at conversion, tax set-aside
Hackathon directions	Payments, fraud alerts	Payments, fraud alerts, private financial intelligence (tax)

Area	v1 (Oct 2 proposal)	v2
Team	Three roles split by layer	Three roles, each owning one agent plus one platform layer
Scope	One flow end to end	Same flow; Wildcard is built only if the happy path is done by Saturday 20:00

The agents

Each agent is the same two halves: hard checks written in code that decide, and a QVAC prompt that explains the result and proposes one action for the merchant to approve.

Fraud Controller: invoice and payment discrepancies

- **Runs on:** a new invoice draft, an incoming RGB payment, a re-imported invoice PDF.
- **Hard checks (code):** line items sum to the total; VAT math; duplicate invoice (same client, amount and date, or near-identical text by embedding); stored hash vs re-imported PDF; asset, exact amount and expiry of each payment; a second payment against an already-paid invoice.
- **Agent's job (QVAC):** compares the event with this client's local history ("5x their usual amount", "first payment ever"), sets a risk level, writes a one-line reason, proposes one action: refund the difference, re-issue, hold, or ask the client.
- **Output:** a flag card with reason, evidence and the proposed action. A failed hard check blocks on its own; the agent can never unblock it.

Commercialista: tax on crypto receipts and conversions

- **Runs on:** a payment marked Paid, a conversion of USDT into EUR, and on demand ("how much should I set aside?").
- **Hard math (code):** the EUR value at receipt from the price feed, stored with the payment as income and as cost basis; on conversion, gain or loss = EUR proceeds − cost basis of the USDT sold; estimated tax from a rates table keyed by year. Rates live in config, never in a prompt.
- **Agent's job (QVAC):** explains each figure in plain Italian or English, warns before a conversion ("converting 500 USDT now realises a gain of about €12, estimated tax €4"), keeps a running "set aside" figure, answers simple questions over the ledger.

- **Output:** a tax line on every ledger row, a pre-conversion warning, and a year-to-date summary. Every figure is labelled as an estimate to confirm with the merchant's accountant.

Tax rules Kasa models for 2026 (as of the date above):

Event	What Commercialista records
USDT received for an invoice	Fee booked at its EUR value at receipt; that value becomes the cost basis of those USDT (our modelling assumption, to confirm)
USDT converted to EUR or spent	Gain or loss taxed at a 33% flat substitute tax from 1 Jan 2026, with no €2,000 exemption any more; the 26% rate is kept only for euro stablecoins (EMTs), so it does not cover USDT
USDT still held on 31 Dec	0.2% annual IVCA on its value, and the holding reported in Quadro RW

Open question: for a freelancer with a partita IVA (forfettario vs ordinario), the USDT may count as a business asset rather than a private holding, which changes how the conversion gain is taxed. Get a 15-minute check from a real commercialista before the pitch; judges will ask.

The conversion itself is simulated in the demo (no EUR off-ramp on testnet): the merchant records "sold X USDT at rate R" and Kasa treats it as a real event. We say so on stage.

Wildcard: third agent slot

The slot is fixed (same interface, same approval card); only its job is open. Decide before the event, suggested by Wed 14 Oct, so its owner can prepare.

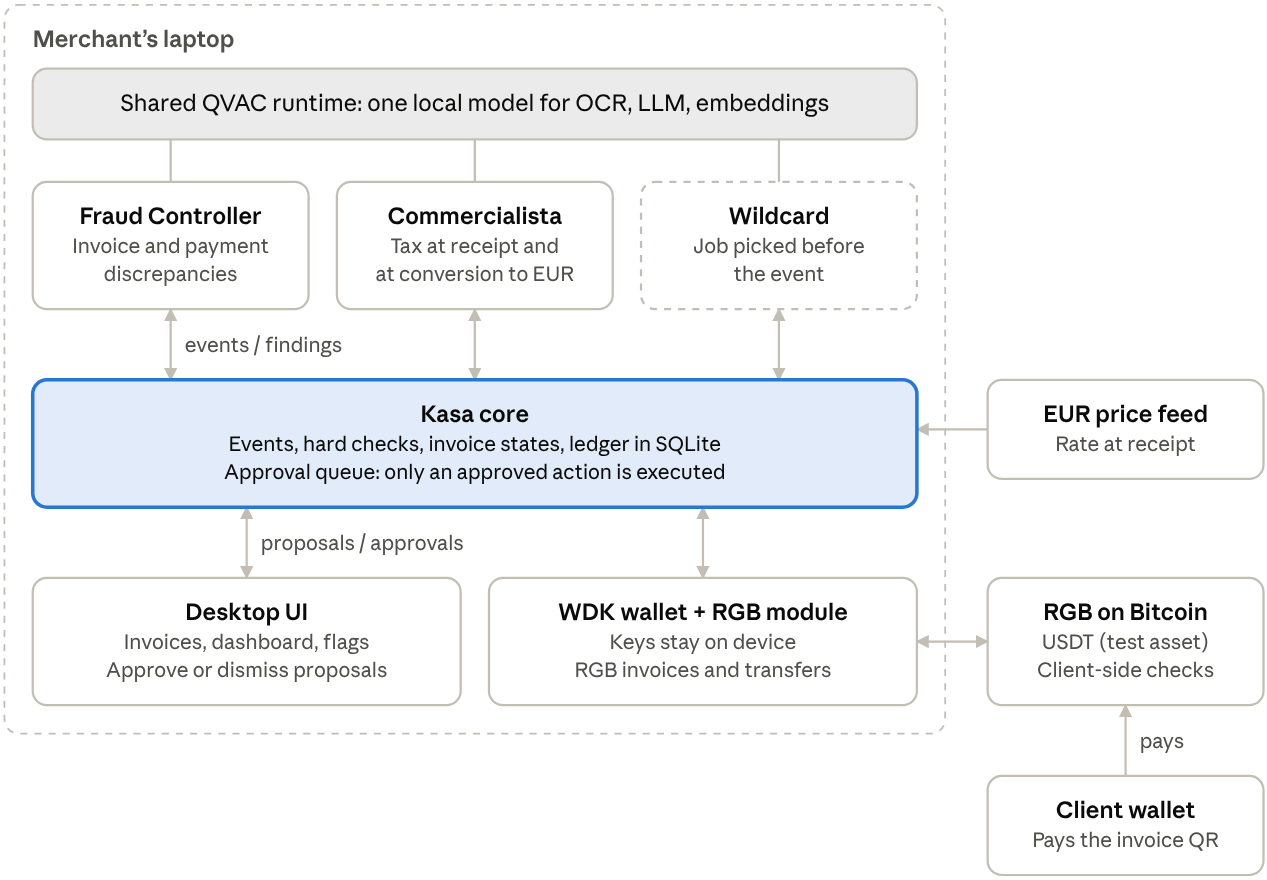
Candidate	Job	Build cost	Why pick it
Collections (recommended)	Chases Expired and Underpaid invoices: drafts reminders, proposes re-issue with a new expiry	Low	Reuses existing invoice states; adds a visible "agent does the boring follow-up" moment

Candidate	Job	Build cost	Why pick it
Treasury	Watches the USDT balance and EUR rate, proposes when and how much to convert	Medium	Pairs with Commercialista's gain estimate; strong finance story
Ledger Q&A	Answers "how much did client X pay in September?" over the local ledger	Low to medium	Shows embeddings and privacy; v1 stretch goal
Payables screener	Screens incoming supplier invoices for fraud before paying	High	Opens a second flow; v1 roadmap item, risky in 30 hours

Architecture

One desktop app on the merchant's laptop: the three agents share one QVAC model and talk only to Kasa core, the only part that changes an invoice's state or calls WDK.

Agents advise Kasa core; only the core changes state or moves money



Kasa architecture · 3 agents on one local runtime

Agents only ever return findings; the approval card and Kasa core sit between every proposal and the wallet.

Stack: Electron + TypeScript; QVAC SDK (OCR, LLM, embeddings) shared by all agents; WDK with the wdk-wallet-rgb module from Utexo; local SQLite for invoices, ledger, tax lots and agent history; RGB on testnet or regtest.

Core flow

The hackathon build is still one receivables flow; the agents hook in at the steps marked below.

1. **Draft.** The merchant drops in an order PDF, photo or email; QVAC extracts client, line items, VAT and total.
   - *Fraud Controller:* totals, VAT and duplicate checks. A mismatch blocks the draft and shows why.

2. **Approve and issue.** The merchant approves; Kasa creates a real RGB invoice for the USDT test asset with amount and expiry, shown as a QR. The invoice hash is stored next to the RGB invoice ID.
3. **Client pays** from a second RGB wallet; the transfer lands in the merchant's WDK wallet.
4. **Reconcile (code only).** Match to invoice; set the state: Issued, Pending, Paid, Underpaid, Overpaid, Expired or Flagged.
   - *Fraud Controller:* payment checks plus history review, then a proposal ("refund 20 USDT") on the approval card.
5. **Book it.** Each Paid invoice becomes a ledger row: date, USDT amount, EUR rate at receipt, EUR value.
   - *Commercialista:* tags the row as income, stores the cost basis, updates the set-aside figure.
6. **Convert (later).** The merchant records a USDT to EUR conversion.
   - *Commercialista:* shows the gain or loss and estimated tax before the merchant confirms.
7. **Follow up.**
   - *Wildcard (if Collections):* Expired or Underpaid invoices get a drafted reminder or a re-issue proposal.

Every proposal from any agent goes through the same approval card; only an approved action is executed, by Kasa core through WDK.

Hackathon scope

About 30 building hours (Saturday 09:00 to Sunday 15:00): two agents are core, the third is earned by hitting the Saturday 20:00 checkpoint.

In scope

- Merchant-side desktop app, receivables only, real RGB transfers on testnet or regtest with a test RGB20 asset standing in for USDT
- Self-custodial WDK wallet with the RGB module
- Shared agent runtime: one QVAC model, one agent interface, one approval card
- Fraud Controller: all hard checks above, plus history-based reasons and proposals
- Commercialista: EUR value and cost basis per payment, simulated conversion with gain and estimated tax, set-aside figure

- Deterministic reconciliation, invoice states and plain-language error messages

Stretch

- Wildcard agent (only if the happy path works by Saturday 20:00)
- Year-to-date tax summary and CSV export for the accountant
- Payments over Lightning via the RGB Lightning Node

Out of scope (roadmap)

- Consumer checkout, holding merchant funds, real EUR off-ramp
- Filing tax, or tax rules for any country other than Italy

Team of three

Each person owns one agent end to end plus one platform layer, all in TypeScript; the agent interface below is the contract between them, agreed before the event.

Role	Platform layer	Agent	Before the event	Done when
A: Payments + fraud	Two WDK wallets with the RGB module, RGB invoices, payment detection, executing approved refunds, testnet or regtest	Fraud Controller (hard checks and proposals)	Run the RGB and WDK tutorials; issue a test RGB20 asset; list the fraud cases to demo	A real payment lands, is reconciled, and an overpayment produces a refund that executes
B: AI runtime + tax	QVAC setup, shared agent runtime and prompts, invoice extraction, price feed with cached rates	Commercialista	Run the QVAC quickstart on the demo laptop; pick a small model; get the tax table checked by a commercialista	Every demo invoice drafts correctly; every paid invoice and conversion shows correct EUR, gain and estimated tax

Role	Platform layer	Agent	Before the event	Done when
C: Product + pitch	Kasa core state machine, ledger, hashing, approval card, UI, demo script and slides	Wildcard	Sketch screens and the state machine; shortlist the Wildcard job and get team sign-off	An outsider runs the demo flow without help

Shared agent contract (Role B writes it first, Saturday 09:00):

```
interface Agent {
  name: 'fraud' | 'commercialista' | 'wildcard';
  triggers: KasaEvent['type'][];          // e.g. 'invoice.drafted',
'payment.received'
  run(event: KasaEvent, ctx: AgentContext): Promise<Finding[]>;
}

interface Finding {
  severity: 'info' | 'warn' | 'block';    // 'block' only from hard checks
  reason: string;                          // one plain-language line
  evidence: Record<string, unknown>;       // numbers the code computed
  proposal?: Action;                       // refund | reissue | remind |
hold
}
```

Everyone watches the RGB, WDK and QVAC tutorials and rehearses the pitch.

Build checkpoints

The RGB round-trip is still the first thing built; nothing agent-related starts on Role A's side until one transfer works.

When	Gate	A: Payments + fraud	B: AI + tax	C: Product + pitch
Sat 13:00	Rails work	One real RGB transfer between the two wallets	QVAC extracts one invoice to JSON; agent contract merged	UI skeleton; state machine in code

When	Gate	A: Payments + fraud	B: AI + tax	C: Product + pitch
Sat 20:00	Happy path	Payment detected and reconciled to Paid	Commercialista books EUR value and cost basis	Draft, approve, QR, Paid on screen. **Go / no-go on Wildcard**
Sun 10:00	Feature freeze	Fraud hard checks, overpay refund executes	Conversion gain and tax estimate; all demo invoices draft correctly	Approval card for all agents; Wildcard if go
Sun 13:00	Bug fixes only	Fix	Fix	Rehearse the demo twice
Sun 15:00	Submit			

Anything not done by Sunday 10:00 is cut, Wildcard first.

Demo script

About three minutes, two wallets on one network, one beat per agent, ending on the privacy line.

1. **Draft.** Drop an order PDF into Kasa; QVAC drafts the invoice, Fraud Controller's checks pass, the merchant approves, an RGB QR appears.
2. **Pay and book.** Pay it from the second wallet with a real RGB transfer. The dashboard flips to Paid; Commercialista adds the ledger row with EUR value and the updated set-aside figure.
3. **Overpay.** Overpay a second invoice. Fraud Controller flags Overpaid and proposes a refund with a one-line reason; approve it and show it execute.
4. **Tamper.** Edit the amount on an issued invoice PDF and re-import it; the hash mismatch is blocked and explained.
5. **Convert.** Record a USDT to EUR conversion at a different rate; Commercialista shows the gain and the estimated 33% tax before the merchant confirms.
6. **Wildcard.** One beat for the third agent (e.g. an expired invoice gets a drafted reminder). Cut first if over time.
7. **Offline.** Turn off Wi-Fi and show all agents still work. Close with: none of this data left the laptop, and Kasa never held the money.

Risks

The RGB payment round-trip is still the biggest risk; the new ones come from having three agents and from giving tax numbers.

Risk	Impact	Mitigation
RGB plus WDK integration harder than expected	No payments, no demo	Tutorials before the event; one real transfer before any UI or agent work
Three agents is too much for 30 hours	Three half-working agents	Agents are prompts plus check functions on one runtime, not three systems; Wildcard only after the 20:00 go
Local model too slow running several agents	Laggy demo	One small quantized model loaded once; agents run one at a time from a queue; demo invoices tested in advance
Tax figures wrong or challenged	Judges doubt Commercialista	Rates in a config table with sources; every figure labelled an estimate; table checked by a real commercialista before the event
Freelancer tax treatment unclear (business vs private holding)	Wrong rule applied on stage	Confirm with the commercialista; if unresolved, say which assumption the demo uses
No EUR off-ramp on testnet	Conversion looks fake	Record the conversion manually and say so openly
Price feed needs network	Breaks the offline beat	Fetch and store the rate at receipt; offline demo uses stored rates
Refunds need an RGB invoice from the payer	Refund can't execute	Second demo wallet provides a refund invoice; in the product, Kasa requests one from the client

Risk	Impact	Mitigation
Agent proposals look arbitrary	Judges doubt the fraud story	Every flag shows its reason and evidence; hard checks do the blocking
"Are you a payment provider?"	Regulatory question	Self-custody by design: Kasa never holds keys or funds

Sources

- Crypto Taxation in Italy: Rules, Thresholds, and History (Waltio, July 2026): 33% rate from 2026, end of the €2,000 threshold, 26% for euro EMTs, IVCA and Quadro RW
- Italy Crypto Tax 2026: Capital Gains Rise to 33% (SpazioCrypto): 2026 Budget Law, EMT exception, DAC8 reporting
- From the v1 proposal: Agentic Dollars Hackathon (BitPolito); Tether launches QVAC SDK (Decrypt); RGB support in WDK via Utexo (Cointelegraph); Will USDt-on-RGB accelerate RGB adoption? (Bitfinex)